DataWiseRE Methodology Report

Fix-and-Flip Screening Pipeline: Formulas, Calculations, and Data Architecture

Generated: April 5, 2026 | Strategy Profile: DENVER_FLIP_V1 | Platform: Next.js + Supabase

Table of Contents

1. Executive Overview
2. System Architecture Map
2.1 End-to-End Data Flow
2.2 Database Schema Summary
2.3 Page & Component Map
3. Comparable Selection & Scoring
3.1 Hard Filters
3.2 Weighted Scoring System
3.3 Haversine Distance
3.4 Assessment
4. ARV Calculations (After Repair Value)
4.1 Per-Comp Adjusted ARV
4.2 Aggregate ARV (Exponential Decay)
4.3 Configuration Parameters
4.4 Assessment
5. Rehab Budget Calculations
5.1 Composite Multiplier System
5.2 Line Item Formulas
5.3 Rate Tables by Property Type
5.4 Assessment
6. Holding Cost Calculations
6.1 Days Held Formula
6.2 Daily Cost Calculations
6.3 Assessment
7. Transaction Cost Calculations
7.1 Cost Formulas
7.2 Assessment
7.5. Financing Cost Calculations
7.5.1 Loan & Cost Formulas

7.5.2 Configuration & Overrides
7.5.3 Assessment
8. Deal Math & Max Offer
8.1 Core Formulas
8.2 Deal Waterfall
8.3 Assessment
9. Prime Candidate Qualification
9.1 Qualification Rules
9.2 Assessment
10. Manual Override System
11. Complete Strategy Profile Reference
12. Cross-Cutting Recommendations

---

1. Executive Overview

DataWiseRE's fix-and-flip screening pipeline automates the initial deal qualification workflow that was previously performed manually in MS Access and Excel. The system processes properties through a six-stage calculation pipeline:

```
MLS Data Import
    |
Comparable Search & Scoring  -- find similar closed sales
    |
ARV Calculation              -- estimate after-repair value
    |
Rehab Budget Estimation      -- estimate renovation costs
    |
Holding & Transaction Costs  -- estimate carry and closing costs
    |
Financing Costs              -- hard money interest + origination
    |
Deal Math & Qualification    -- compute max offer, flag Prime Candidates
```

Design philosophy: The system is designed as an *opportunity identification engine*, not a precision appraisal tool. Its purpose is to rapidly surface properties with favorable deal characteristics so a human analyst can focus their time on the most promising opportunities. Every automated value can be overridden by the analyst.

Engine architecture: All calculation engines are pure functions with no database dependencies. They receive inputs and a strategy profile, and return structured results. The strategy profile (`DENVER_FLIP_V1`) centralizes every configurable parameter, ensuring engines contain zero hardcoded constants.

Source files:

Module	File	Purpose
Comp Scoring	`lib/comparables/scoring.ts`	Weighted multi-factor comparable ranking
ARV Engine	`lib/screening/arv-engine.ts`	Per-comp adjusted ARV with exponential decay aggregation
Rehab Engine	`lib/screening/rehab-engine.ts`	Multiplier-based rehab budget estimation
Holding Engine	`lib/screening/holding-engine.ts`	Time-based carrying costs
Transaction Engine	`lib/screening/transaction-engine.ts`	Title, closing, and commission costs
Financing Engine	`lib/screening/financing-engine.ts`	Hard money loan interest and origination
Deal Math	`lib/screening/deal-math.ts`	Max offer and spread calculations
Qualification	`lib/screening/qualification-engine.ts`	Prime Candidate identification
Strategy Profile	`lib/screening/strategy-profiles.ts`	All configurable parameters
Bulk Runner	`lib/screening/bulk-runner.ts`	Batch orchestrator

2. System Architecture Map

2.1 End-to-End Data Flow

```
CSV Import (REcolorado MLS)
    |
    v
import_batches / import_batch_rows / import_batch_files
    |  (staging + validation + processing)
    v
+-------------------+     +--------------------+     +----------------------+
| real_properties   |<--->| property_physical  |     | property_financials  |
| (identity/loc)    |     | (structure/size)   |     | (tax, HOA)           |
+-------------------+     +--------------------+     +----------------------+
    |                         |
    v                         v
+-------------------+
| mls_listings      |     Canonical property + listing data
| (events/prices)   |
+-------------------+
    |
    |--- SCREENING PATH -------------------------+
    |                                            |
    v                                            v
+--------------------+    +-------------------------------+
| screening_batches  |    | comparable_search_runs        |
| (batch metadata)   |    | comparable_search_candidates  |
+--------------------+    +-------------------------------+
    |                         |
    v                         v
+-------------------------------------------------------------------+
| screening_results                                                 |
| (denormalized: subject snapshot + ARV + rehab + hold + tx +       |
|  deal math + qualification + promotion link)                      |
+-------------------------------------------------------------------+
    |
    |--- PROMOTION TO ANALYSIS -------+
    |                                 |
    v                                 v
+-------------------+     +--------------------+
| analyses          |     | manual_analysis    |
| (scenario shell)  |     | (analyst overrides)|
+-------------------+     +--------------------+
    |
    v
analysis_pipeline / analysis_notes / analysis_reports
```

2.2 Database Schema Summary

Table	Purpose	Key Columns
`real_properties`	Canonical property identity	address, lat/lon, parcel_id, lot_size
`property_physical`	Physical characteristics	sqft (building, above/below grade), beds, baths, year_built, property_type, level_class
`property_financials`	Financial facts	annual_property_tax, annual_hoa_dues
`mls_listings`	MLS listing events	list_price, close_price, close_date, mls_status, property_condition_source
`import_batches`	Import session tracking	source_system, status, row_count, summary_json
`import_batch_rows`	Staged raw CSV rows	raw_row (jsonb), processing_status, validation_errors
`comparable_profiles`	Search parameter templates	purpose, rules_json (tolerances, max distance, etc.)
`comparable_search_runs`	Search execution records	subject IDs, parameters_json, summary_json
`comparable_search_candidates`	Scored comp candidates	distance_miles, raw_score, score_breakdown_json, selected_yn
`comparable_sets`	Curated comp sets	analysis_id, purpose, status
`screening_batches`	Screening run metadata	strategy_profile_slug, total_subjects, prime_candidate_count
`screening_results`	Full screening outputs	All engine outputs denormalized (see Section 8)
`analyses`	Analysis scenario shell	strategy_type, scenario_name, status
`manual_analysis`	Analyst overrides	arv_manual, rehab_manual, target_profit_manual, days_held_manual
`analysis_pipeline`	Workflow tracking	interest_level, showing_status, offer_status

2.3 Page & Component Map

Route / Component	Purpose	Data Sources
`/analysis/properties`	Property browser with filters	`property_browser_v` (view)
`/analysis/properties/[id]`	Property hub	`real_properties` + `property_physical` + `mls_listings`
`/analysis/properties/[id]/analyses/[analysisId]`	Analysis workstation (single-page)	All tables + engine calculations on load
`/analysis/screening`	Screening dashboard	`screening_batches`
`/analysis/screening/[batchId]`	Ranked deal table	`screening_results`
`/analysis/screening/[batchId]/[resultId]`	Full deal detail	`screening_results` + `comparable_search_candidates`
`analysis-workstation.tsx`	Core analysis UI component	Receives pre-computed engine results as props
`comp-map.tsx`	Leaflet map with subject/comp pins	Lat/lon from `real_properties` + comp metrics
`comparable-workspace-panel.tsx`	Comp search & selection panel	`comparable_search_candidates`

3. Comparable Selection & Scoring

Goal: Find the most similar recently-closed sales to a subject property, rank them by overall similarity, and present them for analyst selection. The comp engine is intentionally separated from the valuation engine — its output is a *selected comp set*, not a value.

3.1 Hard Filters (Screening Mode)

Before scoring, candidates must pass all hard filters. Any failure eliminates the candidate.

Filter	Threshold	Source
Max distance	0.75 miles	Haversine from lat/lon
Max days since close	365 days	`mls_listings.close_date`
Sqft tolerance	±30%	`property_physical.building_area_total_sqft`
Lot size tolerance	±30%	`real_properties.lot_size_sqft`
Year built tolerance	±25 years	`property_physical.year_built`
Bed tolerance	±2	`property_physical.bedrooms_total`
Bath tolerance	±2	`property_physical.bathrooms_total`
Same property type	Required	`property_physical.property_type`
Same level class	Required (score ≥ 0.85)	`property_physical.level_class_standardized`
Same building form	Required (exact match)	`property_physical.building_form_standardized`
Max candidates returned	25	Top scores after filtering

3.2 Weighted Scoring System

Candidates passing hard filters receive a weighted composite score (0–100). Each component produces a 0–1 score, then all components are combined using purpose-specific weights.

Component Score Formula

```
componentScore = clamp(0, 1, 1 - delta / tolerance)
```

Where `delta` is the absolute or percentage difference, and `tolerance` is the maximum acceptable difference for that metric.

Special Match Scores

Match Type	Exact Match	Class Match	Partial/Missing	No Match
Building Form	1.0	—	0.45	0.0
Level Class	1.0	0.85	0.45–0.65	0.0
Condition	1.0	—	0.45	0.3

Flip-Mode Scoring Weights

Component	Weight	Input Source
Distance	0.24	Haversine miles
Size (sqft)	0.24	`building_area_total_sqft` % delta
Recency	0.18	Days since `close_date`
Building Form	0.12	`building_form_standardized`
Level Class	0.12	`level_class_standardized`
Year Built	0.10	`year_built` delta
Lot Size	0.08	`lot_size_sqft` % delta
Bedrooms	0.06	`bedrooms_total` delta
Bathrooms	0.06	`bathrooms_total` delta
Condition	0.06	`property_condition_source`

Final Score

```
rawScore = clamp(0, 1, weightedScoreTotal / totalWeight) × 100
```

3.3 Haversine Distance Formula

```
a = sin²(Δlat/2) + cos(lat1) × cos(lat2) × sin²(Δlon/2) c = 2 × atan2(√a, √(1−a)) distance = 3,958.8 × c [miles]
```

Inputs: `real_properties.latitude`, `real_properties.longitude` for both subject and comp.

3.4 Assessment

Strengths:

- Purpose-driven weight profiles (flip vs. rental vs. scrape) tailor comp selection to the analysis strategy.
- Hard filters prevent irrelevant candidates from entering the scoring stage, keeping the comp pool focused.
- Building form and level class matching go beyond basic bed/bath comparison, reflecting real property structure differences.

- Full score breakdown is stored as JSON for transparency and analyst review.

Weaknesses & Suggestions:

- **No neighborhood/subdivision awareness.** Two properties 0.3 mi apart can be in very different neighborhoods. Consider adding subdivision match as a scoring component or allowing analysts to define neighborhood boundaries.
- **Condition scoring is coarse.** Only exact match (1.0) vs. mismatch (0.3). The MLS condition field is often unreliable or missing. Consider supplementing with DOM (days on market) or price-change history as a proxy for condition.
- **No garage/feature adjustment.** Garage count differences can materially affect value. Consider adding garage_spaces as a scoring component.
- **Binary lot size tolerance.** A property at 29% delta passes; 31% is eliminated. Consider a gradual penalty ramp instead of a hard cutoff for fringe candidates.

4. ARV Calculations (After Repair Value)

Goal: Estimate the post-renovation market value of a subject property based on comparable closed sales, adjusting for size differences and time elapsed since each comp closed.

Source: `lib/screening/arv-engine.ts`

4.1 Per-Comp Adjusted ARV

Each comparable produces its own adjusted ARV estimate for the subject, computed in two layers (building-total and above-grade) then blended.

Step 1: Price Per Square Foot

```
psfBuilding = closePrice / compBuildingSqft psfAboveGrade = closePrice /
compAboveGradeSqft
```

Inputs: `mls_listings.close_price`, `property_physical.building_area_total_sqft`, `property_physical.above_grade_finished_area_sqft`

Step 2: Size-Adjusted ARV (Two Layers)

```
arvBuilding = closePrice + (subjectBldgSqft − compBldgSqft) × psfBuilding ×
buildingDampening arvAboveGrade = closePrice + (subjectAboveSqft −
compAboveSqft) × psfAboveGrade × aboveGradeDampening
```

Dampening factors (0–1) reduce the impact of size differences, recognizing that value does not scale linearly with square footage. For detached properties: building dampening = 0.3, above-grade dampening = 0.4.

Step 3: Blended ARV

```
arvBlended = arvBuilding × buildingTotalWeight + arvAboveGrade ×
aboveGradeWeight
```

Property Type	Building Weight	Above-Grade Weight	Building Dampening	Above-Grade Dampening
Detached	0.40	0.60	0.30	0.40
Condo	0.15	0.85	0.20	0.45
Townhome	0.35	0.65	0.30	0.40

Step 4: Time Adjustment

```
daysSinceClose = floor((referenceDate − closeDate) / 86,400,000)
timeMultiplier = 1 + timeAdjustmentAnnualRate × (daysSinceClose / 365)
arvTimeAdjusted = arvBlended × timeMultiplier
```

Current rate: -5% per year. A comp that closed 6 months ago receives a multiplier of 0.975 (2.5% downward adjustment).

Step 5: Confidence Score (Reference Only)

Distance	Confidence
0 – 0.3 mi	1.0 (100%)
0.3 – 0.5 mi	0.8 (80%)
0.5 – 0.6 mi	0.6 (60%)
0.6 – 0.75 mi	0.4 (40%)
> 0.75 mi	0.2 (20%)

> **Note:** Confidence scores are currently stored for analyst reference but are **not used** in the weighted ARV aggregation. The exponential decay weighting uses time, not distance.

4.2 Aggregate ARV (Exponential Decay Weighted Average)

Step 6: Decay Weight

```
decayWeight = e^(−(daysSinceClose / 365))
```

This naturally weights recent comps much more heavily:

Comp Age	Decay Weight	Relative Influence
1 month	0.920	Very high
3 months	0.779	High
6 months	0.607	Moderate
9 months	0.472	Low-moderate
12 months	0.368	Low

Step 7: Weighted Average

```
arvAggregate = Σ(arvTimeAdjusted_i × decayWeight_i) / Σ(decayWeight_i)
arvPerSqft = arvAggregate / subjectBuildingSqft
```

Output stored in: `screening_results.arv_aggregate`, `screening_results.arv_per_sqft`, `screening_results.arv_comp_count`, `screening_results.arv_detail_json`

Downstream consumers: Deal Math engine (max offer calculation), Qualification engine (Prime Candidate check), Analysis workstation UI (3-tier ARV display)

4.3 Configuration Parameters

Parameter	Value	Location
`timeAdjustmentAnnualRate`	-0.05 (-5%/year)	Strategy profile
`confidenceTiers`	5 distance bands	Strategy profile
`blendByPropertyType`	Per-type weights/dampening	Strategy profile

4.4 Assessment

Strengths:

- **Dual-layer valuation** separates building-total from above-grade, properly handling basements and multi-level structures.
- **Dampening factors** prevent extreme size adjustments from distorting value estimates.
- **Exponential decay** is mathematically superior to linear time adjustments — older comps fade smoothly rather than falling off a cliff.
- **Property-type-aware blending** correctly gives condos heavier above-grade weight (no exterior) while detached homes use more balanced weights.

Weaknesses & Suggestions:

- **Fixed -5% annual time adjustment.** This assumes a declining market. In an appreciating market, it understates ARV. Consider deriving this rate from actual market data (e.g., rolling 6-month median price change in the target area).
- **Confidence tiers are computed but unused.** Distance-based confidence could be incorporated as a second weighting dimension (multiply decay weight × confidence) to favor closer comps more strongly.
- **No condition adjustment.** A renovated comp and a dated comp at the same distance receive the same treatment. Consider adding a condition adjustment factor per comp.
- **No feature adjustments beyond size.** Garage, lot size, bed/bath differences are captured in scoring but not in ARV adjustment. These are secondary but could be added as optional refinements.
- **No outlier detection.** A single comp with an extreme price could skew the aggregate. Consider flagging comps whose per-comp ARV deviates >20% from the median.

5. Rehab Budget Calculations

Goal: Estimate the total renovation cost for a fix-and-flip based on property characteristics, using a composite multiplier system that adjusts base per-sqft rates for property type, condition, price tier, and age.

Source: `lib/screening/rehab-engine.ts`

5.1 Composite Multiplier System

Four independent multipliers are resolved and combined into one composite:

```
compositeMultiplier = typeMultiplier × conditionMultiplier × priceMultiplier
× ageMultiplier
```

Type Multiplier

Property Type	Multiplier	Rationale
Detached	1.00	Baseline — full scope of exterior, systems, interior
Townhome	0.95	Reduced exterior/landscaping scope
Condo	0.85	No exterior/landscaping, limited systems

Input: `property_physical.property_type` resolved via `resolvePropertyTypeKey()`

Condition Multiplier

MLS Condition	Multiplier
"fixer"	1.15
"fixer, incomplete"	1.15
"new construction" / "new_construction"	0.00 (zero rehab)
All other / null	1.00 (default)

Input: `mls_listings.property_condition_source`

Price Tier Multiplier (evaluated top-to-bottom, first match wins)

Rule	Multiplier	Rationale
≤ $300,000	0.85	Lower-priced homes get less expensive finishes
≤ $400,000	0.90	
≤ $500,000	0.95	
≥ $900,000	1.20	High-end homes require premium finishes
≥ $700,000	1.10	
$500k–$700k	1.00 (default)	Baseline range

Input: `mls_listings.list_price`

Age Tier Multiplier (evaluated top-to-bottom, first match wins)

Rule	Multiplier	Rationale
Built before 1930	1.25	Older homes: knob-and-tube, lead paint, structural issues
Built before 1950	1.15	Aging systems, possible asbestos
Built after 2019	0.20	Nearly new — minimal rehab needed
Built after 2010	0.40	Modern construction, light cosmetic only
Built after 2005	0.75	Newer but may need cosmetic updates
1950–2005	1.00 (default)	Standard rehab scope

Input: `property_physical.year_built`

5.2 Line Item Formulas

```
aboveGradeInterior = aboveGradeSqft × rate × compositeMultiplier
belowGradeFinished = belowGradeFinishedSqft × rate × compositeMultiplier
belowGradeUnfinished = belowGradeUnfinishedSqft × rate × compositeMultiplier
belowGradeTotal = belowGradeFinished + belowGradeUnfinished interior =
aboveGradeInterior + belowGradeTotal exterior = aboveGradeSqft ×
exteriorRate × compositeMultiplier landscaping = aboveGradeSqft ×
landscapingRate × compositeMultiplier systems = (flatAmount OR buildingSqft
× systemsRate) × compositeMultiplier totalRehab = interior + exterior +
landscaping + systems perSqftBuilding = totalRehab / buildingSqft
perSqftAboveGrade = totalRehab / aboveGradeSqft
```

5.3 Rate Tables by Property Type

Rate	Detached	Condo	Townhome
Above-grade interior	$35/sqft	$35/sqft	$35/sqft
Below-grade finished	$39/sqft	$39/sqft	$39/sqft
Below-grade unfinished	$49/sqft	$49/sqft	$49/sqft
Exterior	$4.40/sqft	$0 (N/A)	$3.30/sqft
Landscaping	$2.70/sqft	$0 (N/A)	$1.50/sqft
Systems	$1.70/sqft bldg	$1,500 flat	$3,000 flat

Output stored in: `screening_results.rehab_total`, individual line item columns, `rehab_composite_multiplier`, `rehab_detail_json`

Downstream consumer: Deal Math engine (`rehabTotal` input)

5.4 Assessment

Strengths:

- **Multiplicative system is elegant and tunable.** Each dimension (type, condition, price, age) acts independently, making it easy to adjust one factor without cascading effects.
- **New construction handling** (multiplier = 0) correctly zero-outs rehab for brand-new homes.
- **Property-type differentiation** in both rates and multipliers reflects real market behavior (condos don't need exterior/landscaping).
- **Separate below-grade finished/unfinished rates** properly account for different renovation scopes.

Weaknesses & Suggestions:

- **Static base rates.** $35/sqft is a Denver-specific number from a point in time. Consider adding a rate inflation adjustment or tying rates to a construction cost index.
- **MLS condition field is unreliable.** Only "fixer" triggers a non-default multiplier. Consider supplementing with photo analysis, DOM, or price-to-comp-ratio as condition proxies.
- **No renovation scope differentiation.** A light cosmetic flip and a full gut-job both start from the same base rate. Consider adding scope tiers (cosmetic / moderate / heavy / gut) that the analyst can select, each with different base rate adjustments.
- **Below-grade unfinished rate ($49/sqft) is higher than finished ($39/sqft).** This reflects the cost of *finishing* unfinished space (not just cosmetic updates). This is correct for a flip that finishes the basement, but could overstate costs if the flip plan leaves the basement as-is. Consider making basement finishing optional.
- **No contingency/overhead factor.** Professional estimators typically add 10–15% for unknowns. Consider adding an optional contingency multiplier.

6. Holding Cost Calculations

Goal: Estimate the carrying costs during the renovation and sale period, including property tax, insurance, HOA, and utilities.

Source: `lib/screening/holding-engine.ts`

6.1 Days Held Formula

```
rawDays = baseDays + (buildingSqft - sqftBaseline) × sqftAdjustmentRate
daysHeld = max(minimumDays, round(rawDays))
```

Parameter	Value	Effect
`baseDays`	190	Starting point for a 2,500 sqft home
`sqftBaseline`	2,500 sqft	Zero-adjustment reference point
`sqftAdjustmentRate`	0.085 days/sqft	Larger homes take longer to renovate and sell
`minimumDays`	67	Floor — even small flips need ~2 months

Example calculations:

Building Sqft	Raw Days	Days Held
1,000	190 + (1000-2500) × 0.085 = 62.5	67 (minimum)
1,500	190 + (1500-2500) × 0.085 = 105	105
2,500	190 + 0 = 190	190
3,500	190 + (3500-2500) × 0.085 = 275	275

6.2 Daily Cost Calculations

```
dailyTax = annualPropertyTax / 365 dailyInsurance = (listPrice ×
insuranceAnnualRate) / 365 dailyHoa = annualHoaDues / 365 dailyUtilities =
(buildingSqft × utilityPerSqftMonthly) / 30 totalHoldingCost = (dailyTax +
dailyInsurance + dailyHoa + dailyUtilities) × daysHeld
```

Cost Component	Source	Parameter
Property Tax	`property_financials.annual_property_tax`	Actual from MLS data
Insurance	`mls_listings.list_price`	0.55% annually
HOA	`property_financials.annual_hoa_dues`	Actual from MLS data
Utilities	`property_physical.building_area_total_sqft`	$0.08/sqft/month

Output stored in: `screening_results.hold_total`, `screening_results.hold_days`

Downstream consumer: Deal Math engine (`holdTotal` input)

6.3 Assessment

Strengths:

- **Size-scaled holding period** is a practical innovation — larger homes genuinely take longer to renovate and sell.
- **Real data where available** (actual tax and HOA from MLS) is better than estimates.
- **Minimum floor** prevents unrealistically short hold periods for small properties.

Weaknesses & Suggestions:

- **No financing costs.** Most flippers use hard money loans (10–14% annual interest). This is the single largest missing cost component. The `financingEnabled` flag exists in the profile but is not yet implemented. Prioritize this.
- **Insurance based on list price, not ARV.** Insurance during renovation is typically based on the intended completed value or purchase price. Using list price is reasonable for screening but could understate costs for properties listed below market.
- **No seasonal adjustment.** Properties listed in winter may take longer to sell. Consider adding a seasonal factor or simply flagging this as an analyst consideration.
- **Tax/HOA default to zero when null.** If financial data is missing from the MLS, the holding cost is understated. Consider using a market-average fallback for missing tax data.
- **Utility rate is flat.** $0.08/sqft/month is a rough average. Vacant properties during renovation use less but may have minimum service charges.

7. Transaction Cost Calculations

Goal: Estimate title/closing costs on both the buy and sell side, plus agent commissions on disposition.

Source: `lib/screening/transaction-engine.ts`

7.1 Cost Formulas

```
acquisitionTitle = listPrice × acquisitionTitleRate (0.3%) dispositionTitle
= listPrice × dispositionTitleRate (0.47%) dispositionCommission = ARV ×
dispositionCommissionRate (4.0%) totalTransactionCost = acquisitionTitle +
dispositionTitle + dispositionCommission
```

Cost	Rate	Basis	Input Source
Acquisition title/closing	0.30%	List price (purchase proxy)	`mls_listings.list_price`
Disposition title/closing	0.47%	List price (purchase proxy)	`mls_listings.list_price`
Disposition commissions	4.00%	ARV (expected sale price)	ARV engine output

Output stored in: `screening_results.transaction_total`

7.2 Assessment

Strengths:

- Clean separation of acquisition vs. disposition costs.
- Commission based on ARV (the expected sale price) rather than list price is correct.
- Rates are configurable in the strategy profile.

Weaknesses & Suggestions:

- **Disposition title should be based on ARV, not list price.** Title and closing costs on the sell side are a function of the sale price (ARV), not the original purchase price. This currently understates disposition closing costs.
- **No transfer tax / recording fees.** Colorado has relatively low transfer taxes, but some jurisdictions charge additional fees. Consider adding a per-transaction flat fee component.
- **Commission rate may be high post-NAR settlement.** The 4% disposition commission assumes the seller pays the buyer's agent. Post-2024 NAR settlement, this structure is changing. Consider separating listing agent commission (2–3%) from buyer agent commission (negotiable).
- **No acquisition commission.** If buying through an agent, there may be buyer-side commission or fees. Consider adding an optional acquisition commission rate.

7.5 Financing Cost Calculations

Goal: Estimate hard money loan costs for the flip, including interest carry during the hold period and upfront origination fees. The loan is sized against ARV (not purchase price), which is standard for hard money lending and breaks the circular dependency between financing costs and offer price.

Source: `lib/screening/financing-engine.ts`

7.5.1 Loan & Cost Formulas

```
loanAmount = ARV × ltvPct interestCost = round(loanAmount × annualRate ×
daysHeld / 365) originationCost = round(loanAmount × pointsRate)
totalFinancing = interestCost + originationCost -- Reference values --
monthlyPayment = loanAmount × annualRate / 12 (interest-only) dailyInterest
= loanAmount × annualRate / 365
```

Input	Source	Default
ARV	ARV engine output (effective ARV)	Computed
Days Held	Holding engine output	Computed from sqft formula
Annual Rate	Strategy profile or analyst override	11%
Origination Points	Strategy profile or analyst override	1%
LTV %	Strategy profile or analyst override	80%

Example Calculation

```
ARV                     = $450,000
LTV                     = 80%
Loan Amount             = $450,000 × 0.80 = $360,000

Annual Rate             = 11%
Hold Period             = 190 days
Interest Cost           = $360,000 × 0.11 × (190/365) = $20,614

Points Rate             = 1%
Origination Cost        = $360,000 × 0.01 = $3,600

Total Financing Cost    = $20,614 + $3,600 = $24,214

Monthly Payment (I/O)   = $360,000 × 0.11 / 12 = $3,300
Daily Interest          = $360,000 × 0.11 / 365 = $108.49
```

7.5.2 Configuration & Overrides

Parameter	Profile Default	Override Field	Stored In
Annual Rate	0.11 (11%)	`financing_rate_manual`	`manual_analysis`
Origination Points	0.01 (1%)	`financing_points_manual`	`manual_analysis`
LTV %	0.80 (80%)	`financing_ltv_manual`	`manual_analysis`
Enabled	`true`	—	Strategy profile

Override priority: analyst override → strategy profile default. Users enter values as percentages (e.g. "11" for 11%), stored as decimals (0.11).

Output stored in: `screening_results.financing_total`, `financing_interest`, `financing_origination`, `financing_loan_amount`, `financing_detail_json`

Downstream consumer: Deal Math engine (`financingTotal` input)

7.5.3 Assessment

Strengths:

- **ARV-based loan sizing** is the correct approach for hard money — it breaks the circular dependency and matches how lenders actually underwrite.
- **All three key parameters are overridable** per deal, allowing analysts to model different financing scenarios.
- **Interest-only model** is appropriate for short-term hard money (flippers don't make principal payments).
- **Origination included as a one-time upfront cost** correctly reflects how points work in practice.

Weaknesses & Future Considerations:

- **No draw schedule / rehab holdback modeling.** Many hard money lenders fund rehab in draws, meaning not all loan proceeds are disbursed day one. Interest on rehab draws could be modeled separately for more accuracy.
- **No prepayment penalty modeling.** Some hard money loans have minimum interest periods (e.g., 3 months guaranteed interest). Consider adding a minimum interest floor.
- **Cash-to-close not yet computed.** Consider adding: `cashToClose = purchasePrice - loanAmount + originationCost + acquisitionTitle` to show the equity requirement.
- **No comparison with cash purchase.** Consider showing financing vs. all-cash deal economics side-by-side to help analysts evaluate leverage decisions.

8. Deal Math & Max Offer

Goal: Compute the maximum acceptable purchase price that achieves the target profit, and measure the opportunity gap between ARV and list price.

Source: `lib/screening/deal-math.ts`

8.1 Core Formulas

```
totalCosts = rehabTotal + holdTotal + transactionTotal + financingTotal
maxOffer = ARV − totalCosts − targetProfit spread = ARV − listPrice offerPct
= maxOffer / listPrice estGapPerSqft = spread / buildingSqft
```

Output	Formula	Stored In	Meaning
`totalCosts`	rehab + hold + transaction + financing	Derived	All-in cost to renovate and sell
`maxOffer`	ARV - costs - profit	`screening_results.max_offer`	Highest price to pay and still hit profit target
`spread`	ARV - list price	`screening_results.spread`	Raw gap between value and asking price
`offerPct`	maxOffer / listPrice	`screening_results.offer_pct`	Offer as % of asking (e.g., 0.72 = 72%)
`estGapPerSqft`	spread / buildingSqft	`screening_results.est_gap_per_sqft`	Normalized opportunity measure ($/sqft)

Default target profit: $40,000 (configurable, overridable per analysis via `manual_analysis.target_profit_manual`)

8.2 Deal Waterfall (as displayed in UI)

```
  Effective ARV           $450,000
- Rehab                  - $62,000
- Holding                -  $8,500
- Transaction            - $19,800
- Financing              - $24,214   (11% on $360k loan, 190 days + 1 pt)
- Target Profit          - $40,000
  ──────────────────────────────
= Max Offer              $295,486

  Spread (ARV - List)    = $450,000 - $385,000 = $65,000
  Offer %                = $295,486 / $385,000 = 76.7%
  Gap/sqft               = $65,000 / 1,800 sqft = $36.11/sqft
```

8.3 Assessment

Strengths:

- **Clean, transparent waterfall** makes it easy for analysts to see exactly how the max offer is derived.
- **Target profit is overridable** per deal, allowing flexibility for different risk profiles.
- **Gap/sqft normalization** enables apples-to-apples comparison across different-sized properties.

Weaknesses & Suggestions:

- **Target profit is a flat dollar amount.** Consider offering a percentage-of-ARV option (e.g., 10% of ARV) as an alternative, since larger deals typically require larger absolute returns.
- **No ROI or annualized return calculation.** $40,000 profit on a $320,000 investment over 190 days is very different from $40,000 on a $150,000 investment over 90 days. Consider adding: `ROI = profit / totalInvestment` and `annualizedROI = ROI × (365 / daysHeld)`.

9. Prime Candidate Qualification

Goal: Identify properties with the strongest deal characteristics — those where multiple nearby, recent comps independently support a significant value gap.

Source: `lib/screening/qualification-engine.ts`

9.1 Qualification Rules

A property earns Prime Candidate status when enough individual comparables independently indicate a strong deal. Each comp must pass all three filters:

```
For each comp: compGap = (comp.arvTimeAdjusted - listPrice) / buildingSqft
passesDistance = comp.distanceMiles ≤ 0.4 passesRecency =
comp.daysSinceClose ≤ 213 (~7 months) passesGap = compGap ≥ $60/sqft
isQualifying = passesDistance AND passesRecency AND passesGap
isPrimeCandidate = (count of qualifying comps) ≥ 2
```

Threshold	Value	Rationale
Max comp distance	0.4 miles	Tighter than screening (0.75 mi) — requires very local evidence
Max comp age	213 days (~7 months)	Only recent market data
Min gap per sqft	$60/sqft	Meaningful profit potential
Min qualifying comps	2	Requires independent confirmation from multiple comps

Output: `screening_results.is_prime_candidate` (boolean), `screening_results.qualification_json` (detailed reasons and disqualifiers)

9.2 Assessment

Strengths:

- **Multi-comp confirmation** is the key innovation — requiring 2+ independent qualifying comps dramatically reduces false positives.
- **Per-comp gap analysis** (not just aggregate) prevents a single outlier comp from triggering Prime status.
- **Tighter distance/recency requirements** than general screening ensure Prime Candidates have strong local evidence.
- **Stored reasons and disqualifiers** give the analyst full transparency.

Weaknesses & Suggestions:

- **$60/sqft gap threshold is market-specific.** In a lower-cost market, $60/sqft may be unrealistic. Consider making this a percentage of comp ARV (e.g., 10%) as an alternative or supplementary threshold.
- **No consideration of deal feasibility.** A property with a $60/sqft gap but an ARV of $200,000 may not be worth flipping due to thin absolute margins. Consider adding a minimum absolute spread check.
- **Time since listing not considered.** A property listed yesterday vs. 90 days ago has different negotiation dynamics. Stale listings may offer better negotiation opportunities.

10. Manual Override System

The analysis workstation implements a 3-tier priority system for key values, allowing analysts to override automated calculations at any point.

Effective Value Waterfall

```
effectiveARV = manualARV ?? selectedCompARV ?? autoScreeningARV ?? 0
effectiveRehab = manualRehab ?? computedRehab ?? autoScreeningRehab ?? 0
effectiveTargetProfit = manualTargetProfit ?? profileDefault ($40,000)
```

Override Fields

Override Field	Stored In	What It Overrides
Final ARV	`manual_analysis.arv_manual`	Computed ARV from comps
Rehab Budget	`manual_analysis.rehab_manual`	Computed rehab estimate
Days Held	`manual_analysis.days_held_manual`	Computed days held
Target Profit	`manual_analysis.target_profit_manual`	Profile default ($40k)
Monthly Rent	`manual_analysis.rent_estimate_monthly`	N/A (manual only currently)
Analyst Condition	`manual_analysis.analyst_condition`	MLS condition field
Design Rating	`manual_analysis.design_rating`	N/A (qualitative)
Location Rating	`manual_analysis.location_rating`	N/A (qualitative)

When any override is saved, the page recalculates all downstream values (rehab, holding, transaction, deal math) on the next server-side load, incorporating the override into the calculation chain.

11. Complete Strategy Profile Reference: DENVER_FLIP_V1

General	
Slug	`denver_flip_v1`
Strategy Type	Flip
Target Profit Default	$40,000
ARV	
Time Adjustment Rate	-5% per year
Confidence: 0–0.3 mi	1.0
Confidence: 0.3–0.5 mi	0.8
Confidence: 0.5–0.6 mi	0.6
Confidence: 0.6–0.75 mi	0.4
Confidence: >0.75 mi	0.2
ARV Blending (Detached / Condo / Townhome)	
Building Weight	0.40 / 0.15 / 0.35
Above-Grade Weight	0.60 / 0.85 / 0.65
Building Dampening	0.30 / 0.20 / 0.30
Above-Grade Dampening	0.40 / 0.45 / 0.40
Rehab Base Rates ($/sqft)	
Above-Grade Interior	$35 / $35 / $35
Below-Grade Finished	$39 / $39 / $39
Below-Grade Unfinished	$49 / $49 / $49
Exterior	$4.40 / $0 / $3.30
Landscaping	$2.70 / $0 / $1.50
Systems	$1.70/sqft / $1,500 flat / $3,000 flat
Rehab Multipliers	
Type (D/C/T)	1.00 / 0.85 / 0.95
Condition: fixer	1.15
Condition: new construction	0.00
Price: ≤$300k / ≤$400k / ≤$500k	0.85 / 0.90 / 0.95
Price: ≥$900k / ≥$700k	1.20 / 1.10
Age: <1930 / <1950	1.25 / 1.15
Age: >2019 / >2010 / >2005	0.20 / 0.40 / 0.75
Holding	

Base Days	190
Sqft Baseline	2,500 sqft
Sqft Adjustment Rate	0.085 days/sqft
Minimum Days	67
Insurance Annual Rate	0.55% of list price
Utility Rate	$0.08/sqft/month
Transaction	
Acquisition Title	0.30% of list price
Disposition Title	0.47% of list price
Disposition Commission	4.0% of ARV
Financing (Hard Money)	
Enabled	Yes
Annual Rate	11%
Origination Points	1%
LTV (of ARV)	80%
Qualification (Prime Candidates)	
Max Comp Distance	0.4 miles
Min Gap/sqft	$60/sqft
Max Comp Age	213 days (~7 months)
Min Qualifying Comps	2

12. Cross-Cutting Recommendations

12.1 High Priority

#	Recommendation	Impact	Affected Engines
1	✓ **Financing costs implemented.** Hard money at 11% with 1 point origination, 80% LTV of ARV. All three parameters overridable per deal. Adds $15k–$25k+ to cost basis, correcting the prior max offer overstatement.	Resolved	Financing, Deal Math
2	**Incorporate confidence tiers into ARV weighting.** Multiply `decayWeight × confidence` so that both proximity and recency influence the aggregate. Currently distance-based confidence is computed but unused.	More accurate ARV, especially with distant comps	ARV
3	**Dynamic market time adjustment.** Replace fixed -5% with a rolling market trend rate derived from actual comp data in the area (e.g., median price change over trailing 6 months).	Prevents systematic ARV bias in appreciating or rapidly declining markets	ARV

12.2 Medium Priority

#	Recommendation	Impact	Affected Engines
4	**Add ROI and annualized return metrics.** `ROI = targetProfit / (maxOffer + rehabTotal)` and `annualizedROI = ROI × 365 / daysHeld`. These are essential for comparing deals of different sizes and durations.	Better deal comparison for portfolio decisions	Deal Math
5	**Add rehab scope tiers.** Allow analyst to select cosmetic / moderate / heavy / gut renovation scope, each with a multiplier on base rates (e.g., 0.6 / 1.0 / 1.4 / 2.0).	Significant improvement in rehab accuracy	Rehab
6	**Fix disposition title cost basis.** Should be based on ARV (sale price), not list price (purchase price). Small dollar impact but conceptually incorrect.	Marginal cost accuracy improvement	Transaction
7	**Add market-average fallback for missing tax data.** When `annual_property_tax` is null, use a configurable area default (e.g., 0.6% of list price for Denver) rather than $0.	Prevents understated holding costs	Holding

12.3 Future Enhancements

#	Recommendation	Impact
8	**Neighborhood/subdivision scoring.** Add subdivision match and school district as comp scoring components.	Better comp relevance
9	**Garage/feature adjustment in ARV.** Add garage_spaces delta as an ARV adjustment factor (e.g., $15k–$25k per garage space difference).	Reduced ARV noise from feature mismatches
10	**Outlier detection in ARV.** Flag comps whose per-comp ARV deviates >20% from the median. Alert the analyst but don't auto-exclude.	Analyst awareness of potentially skewed data
11	**Construction cost index tie-in.** Link base rehab rates to a published construction cost index (e.g., RSMeans) with periodic updates.	Rate accuracy over time
12	**Percentage-based target profit option.** Offer 10–15% of ARV as an alternative to flat $40k, better suited for higher-value deals.	Appropriate profit targets across price ranges

12.4 Architecture Strengths Worth Preserving

- **Pure function engines** with no DB dependencies make testing, iteration, and strategy profile experimentation trivial.
- **Single strategy profile as source of truth** for all parameters prevents scattered magic numbers.
- **Screening separate from analysis** keeps the funnel clean — automated results don't pollute analyst-reviewed work.
- **3-tier override system** (auto → computed → manual) gives analysts full control while preserving automation as a starting point.
- **Denormalized screening results** enable fast dashboard queries and sorting without re-running calculations.
- **Comparables engine separate from valuation engine** is architecturally correct and will scale well as more strategy types are added.

DataWiseRE Methodology Report | Generated April 5, 2026
Strategy Profile: DENVER_FLIP_V1 v1 | Engine Source: lib/screening/*.ts + lib/comparables/scoring.ts